

## EXPERIMENT NO 4

# Develop a Map Application Using Dart

### Aim

To install the Flutter/Dart development environment and develop a mobile application that displays an interactive map, shows the user's current location, and places markers on the map using the Google Maps platform.

### Objectives

- Install and configure the Flutter SDK and a supporting IDE.
- Obtain and configure a Maps API key.
- Add the required map and location packages and permissions to a Dart/Flutter project.
- Design a screen that displays a map, the current location, and custom markers.
- Run and test the map application on an emulator or physical device.

### Software / Hardware Requirements

| Requirement      | Details                                                                      |
|------------------|------------------------------------------------------------------------------|
| Operating System | Windows 10/11, macOS, or Linux (64-bit)                                      |
| RAM              | 8 GB minimum (16 GB recommended)                                             |
| Disk Space       | At least 10 GB free                                                          |
| IDE              | Android Studio or Visual Studio Code                                         |
| SDK              | Flutter SDK, Dart SDK (bundled with Flutter)                                 |
| Emulator/Device  | Android Emulator with Google Play services, or a physical Android/iOS device |
| Internet Access  | Required for downloading packages and loading map tiles                      |
| Google Account   | Required to create a Google Cloud project and Maps API key                   |

## Part 1: Introduction to Map Applications in Dart

Dart itself does not provide built-in mapping capability; mapping features are added to a Dart/Flutter project through external packages published on pub.dev, the official Dart and Flutter package repository. The most widely used package for this purpose is the Google Maps plugin for Flutter, which wraps the native Google Maps SDK for Android and iOS and exposes it as a Flutter widget.

A typical map application allows the user to view a map centered on a chosen location, zoom and pan around it, see their own current position (obtained through the device's location services), and view markers, labels, or routes placed on top of the map.

### Key Concepts to be used in this Experiment

- Map Widget: A widget that renders an interactive map inside the app's layout, similar to how other Flutter widgets render text or images.
- Camera Position: The initial location, zoom level, tilt, and bearing used to focus the map when it first loads.
- Markers: Pins placed on the map to highlight specific coordinates, such as the user's location or a point of interest.
- Geolocation: The process of determining the device's current latitude and longitude using its GPS or network location services.
- API Key: A unique credential issued by the Google Cloud Platform that authorizes the app to load map data.
- Permissions: Explicit access to location and internet features that must be declared and granted for the map and geolocation features to function.

## Part 2: Installation of the Flutter/Dart Development Environment

The steps below describe installation on Windows. The same broad steps apply to macOS and Linux, with only the download package and PATH configuration differing.

### Step 1: Download the Flutter SDK

1. Open a web browser and go to the official Flutter website.
2. Select the appropriate operating system and download the latest stable release of the Flutter SDK.
3. Extract the downloaded archive to a simple folder location that does not require elevated privileges.

### Step 2: Update the System PATH Variable

4. Open the system environment variable settings.
5. Locate the "Path" variable under user variables and choose to edit it.
6. Add a new entry pointing to the "bin" folder inside the extracted Flutter SDK folder.
7. Save the changes and close the settings windows.

### Step 3: Install Android Studio and Required Components

8. Download and install Android Studio from its official website.

9. Complete the setup wizard using the default options.
10. Open the SDK Manager and confirm that the Android SDK, Command-line Tools, and Build-Tools are installed.
11. Accept the Android SDK licenses when prompted.

#### Step 4: Install the Flutter and Dart Plugins

12. Open the IDE's plugin or extension manager.
13. Search for and install the Flutter plugin; the Dart plugin will be installed automatically alongside it.
14. Restart the IDE if prompted.

#### Step 5: Set Up an Emulator with Google Play Services

15. Open the Device Manager in Android Studio.
16. Create a new virtual device and choose a phone definition.
17. Select a system image that includes Google Play services, since the Maps SDK depends on Google Play services being available.
18. Finish the setup and confirm the emulator appears in the device list.

#### Step 6: Verify the Installation

Open a terminal and run the Flutter environment check command. This inspects the installation and reports the status of the Flutter SDK, connected devices, and IDE plugins. Any item marked as incomplete should be resolved, such as accepting outstanding Android licenses, before proceeding.

### Part 3: Obtaining a Google Maps API Key

Access to Google's map data requires an API key tied to a Google Cloud project. The following procedure describes how to obtain one.

19. Sign in to the Google Cloud Console using a Google account.
20. Create a new project, or select an existing one, to associate with the map application.
21. Open the API Library and enable the "Maps SDK for Android" (and "Maps SDK for iOS" if targeting iOS).
22. Open the Credentials section and create a new API key.
23. Copy the generated API key and store it securely; it will be needed while configuring the project.
24. Optionally, restrict the API key to the specific app package name and SDKs to prevent unauthorized use.

**Note:** Billing must be enabled on the Google Cloud project for the Maps SDK to function, although Google provides a recurring free usage allowance that is normally sufficient for development and learning purposes.

### Part 4: Creating the Project and Adding Map Support

#### Step 1: Create a New Flutter Project

25. Open a terminal and navigate to the desired working folder.
26. Use the Flutter project-creation command to generate a new project named, for example, "map\_demo\_app".

27. Open the newly created project folder in the IDE.

## Step 2: Add the Required Packages

Map applications depends on external packages that must be declared as dependencies of the project before they can be used.

- A Google Maps package to render the interactive map widget.
- A location or geolocation package to read the device's current GPS coordinates.
- A permission-handling package, if the chosen location package does not manage runtime permissions on its own.

These dependencies are added to the project's configuration file and then fetched by instructing Flutter to retrieve all declared packages before running the app.

## Step 3: Configure the API Key in the Native Project

28. Open the Android-specific configuration file inside the android folder of the project.

29. Insert the Google Maps API key obtained earlier into the designated metadata entry for the Maps SDK.

30. For an iOS target, insert the same API key into the corresponding iOS application delegate configuration.

## Step 4: Declare the Required Permissions

31. Open the Android manifest file and add permissions for internet access and fine and/or coarse location access.

32. For an iOS target, open the Info.plist file and add the usage description keys explaining why the app needs the user's location.

**Note:** Without declaring the internet and location permissions, the map may fail to load or the app may crash when attempting to read the device's current position.

# Part 5: Designing the Map Screen

This section describes, step by step, how the map screen of the application is planned and assembled. No source code is included here; the procedure focuses on what each part of the screen is responsible for and how the pieces fit together.

## Step 1: Plan the Screen Layout

- A top app bar showing the title of the application, such as "Map Demo App".
- A full-screen interactive map occupying the main body of the screen.
- A floating action button positioned over the map to let the user re-center the view on their current location.
- An optional bottom panel or dialog that displays details about a marker when it is tapped.

## Step 2: Set the Initial Camera Position

Decide on a starting location and zoom level for the map when the screen first appears, for example the user's city or campus. This becomes the initial camera position that the map widget uses before the user's live location is available.

### Step 3: Request Location Permission and Fetch Current Location

33. When the screen loads, check whether location permission has already been granted.
34. If not granted, prompt the user with the system permission dialog requesting access to their location.
35. Once permission is granted, request the device's current latitude and longitude from the location service.
36. Update the map's camera position to move and zoom towards the retrieved coordinates.

### Step 4: Display the User's Location and Markers

- Enable the map's built-in "my location" indicator so the user's position is shown as a blinking dot on the map.
- Add a marker at the user's current coordinates with a title, such as "You are here".
- Add one or more additional markers at points of interest, each with a title and short description shown when tapped.
- Group all markers into a single collection that is supplied to the map widget so they are all rendered together.

### Step 5: Handle User Interaction

- Tapping a marker opens an information window showing its title and description.
- Tapping the floating action button re-centers and re-zooms the camera on the current location.
- Dragging or pinch-zooming the map pans and zooms the view, which the map widget handles automatically.

### Step 6: Review the Screen Before Running

- Confirm the API key has been inserted correctly in the native configuration.
- Confirm the required permissions are declared in the manifest / Info.plist.
- Confirm the map widget is placed inside the body of the scaffold and given the initial camera position and marker set.
- Confirm the floating action button and any marker-tap handling are wired to their respective functions.

## Part 6: Running the Application

37. Launch the configured emulator (with Google Play services) or connect a physical device with USB debugging and location services enabled.
38. Fetch the project's declared packages so all map and location dependencies are downloaded.
39. Run the application from the terminal or by using the IDE's run button.
40. When prompted on the device, grant the app permission to access the device's location.
41. Confirm that the map loads, centers on the current location, and displays the "you are here" marker along with any additional points of interest.
42. Tap a marker to confirm its information window appears, and tap the floating action button to confirm the camera re-centers.

## Part 7: Result

A map application was successfully installed, configured, and executed. The application displayed an interactive Google Map, determined and displayed the device's current location, and placed and responded to taps on custom markers, confirming correct integration of the mapping, location, and permission components.

## Part 8: Conclusion

Through this experiment, the complete workflow for building a location-aware application in Dart was studied and practiced: installing and verifying the Flutter/Dart environment, provisioning a Google Maps API key, adding and configuring the required map and location packages, declaring the necessary permissions, and assembling a map screen that displays the current location and custom markers. This provides the foundation for building richer location-based features such as route drawing, geofencing, or place search.